



Опрос про ИИ внутри разработки

[Войти](#)

za-ek2

5 часов назад

200 рублей, ESP12 и полная потеря самоконтроля: хроники одной вентиляции



22 мин



5.4K

Arduino*, Электроника для начинающих, DIY или Сделай сам, Программирование микроконтроллеров*, Умны

[Кейс](#)

Я решил поделиться своим опытом и наработками, вначале небольшой full-up, как я делаю устройство на базе этих наработок, а далее расскажу пошагово, что необходимо, что бы с нуля сделать устройство на базе esp12 и готовых модулей.

Почему esp01 и esp12? Компактно, удобно программировать, на него можно поставить веб-сервер, обновить по воздуху. Приятная цена - один модуль стоит в районе 100 рублей, модуль питания стоит 70 рублей, далее любой обвес от 50 рублей и выше - исполнительные элементы, датчики, sim-карта, RFC, кнопки, дисплеи и прочее. В сумме, за 200 рублей можно получить развлечений на всё свободное время.

Платы я не травлю, вырезал/сверлил на чпу станке, есть много наработок для перевода гербера в рабочие команды. Корпуса - самая муторная часть, готовые из пластика - неудобные, некрасивые и дорогие, из металла - можно было гнуть, но не везде подходят, из дерева - ха, печатные на 3д-принтере - тут есть вопросы по материалам - насколько они устойчивы в агрессивной среде, плюс сложности с эксгумацией рабочих настроек, если печатаешь не каждый день и конечно же долгая печать с запахом.

Как я создаю устройство

1. Определяюсь с моделью esp - сначала пробую сделать с esp01, несколько циклов принятия и перехожу на esp12, несмотря на то, что разница в цене минимальна, esp01

занимает меньше места и есть удобные посадочные места, поэтому я предпочитаю её. У esp01 четыре свободных пина, есть ситуации когда можно использовать их все, в большинстве случаев можно использовать только один, это правило избавляет от сильной головной боли

2. Модули для решения задачи: для esp01 есть модули с готовым посадочным местом, просто вставляешь туда esp и никаких проводов, некоторым модулям нужно согласование питания, иногда нужно сделать дополнительную плату, если её проектировать как модуль - то сохраняется вся концепция создания устройства
3. Программное обеспечение: я написал базовый класс, в котором описал методы обновления, хранения конфига и базовые настройки - подключение к WiFi, fallback в точку доступа, интерфейс установки обновления. Есть небольшое неудобство в хранении библиотеки, так я потерял свою предыдущую версию в связи со смертью жёсткого диска, подключив библиотеку как зависимость для Arduino IDE. Поскольку сборка программы arduino это сборка c++ приложения, то путь к библиотекам надо прописывать. Это можно сделать через подключение внешней библиотеки или положить файлы библиотеки в корень вместе с файлом скетча .ino, тогда они автоматически прилинкуются. Я создаю в репозитории отдельную директорию для каждого проекта, в нём размещаю файлы проекта и создаю символическую ссылку на основную библиотеку.
4. Собираю схему на макетной плате, тестирую нормальную работу, работу при перебое напряжения. Может потребоваться большой ток, например для движения 28BYJ-48 нужен 1A, обычные платы питания рассчитаны на 0.7A, в таком случае произойдёт просадка и модуль зависнет, для таких сценариев я использую дополнительный источник питания (на практике ESP в пике потребляет 0.4A, хоть по даташиту и 0.25A), соответственно, проверять нужно на том источнике, который будет использоваться далее.
5. Отрисовываю схему в KiCAD: в библиотеке есть все необходимые компоненты, включая esp01 и esp12, разве что 3D-модели есть не у всех. Базовая схема - это сам модуль, коннектор Conn_01x02_Pin, компоненты PSpice +3.3V и GND, резистор 3.3/10к для подключения пина RST, пин EN напрямую на 3.3V. Добавляю остальные компоненты через коннекторы, либо непосредственно на плату.
6. KiCAD PCB editor для создания платы: использую модели тех компонентов которые у меня на руках, можно было бы и smd, но как правило элементов нужно мало и резисторы заодно играют роль перемычек, иногда приходится менять footprint, делать соответствующее изменение в схеме и импортировать в PCB Editor. В режиме 3D просмотра перепроверяю правильную распиновку ESP. Всю плату заливаю GND и на слое Edge.Cuts обозначаю границы. Измеряю с помощью Interactive Offset Tool размер edge.cuts для проектирования корпуса.
7. FreeCAD для проектирования 3D-модели. С помощью штангенциркуля определяю высоту элементов на плате и делаю 3D-модель схемы. Модели основных модулей,

как правило, есть, если их нет - они легко прототипируются с помощью того же штангенциркуля. Корпус по классической модели электронных устройств - основание, которое крепится к поверхности и крышка, которая крепится к основанию. Можно использовать разные способы крепления, а для себя я определил как самый удобный - винт-гайка М2, М3 или М4. Для крепления модулей к основанию лучше всего использовать защёлки: размеры печатных плат всегда приблизительные, не во всех есть отверстия для крепления, поэтому, что бы не пилить их напильником, проще защёлкнуть/прижать.

8. Печать на 3D-принтере, гравировка платы, пайка, сборка и установка на место.

Постоянные ошибки, которые преследуют меня

ПО

Изначально я поставил автообновление и набросал небольшой сервер, который отвечает в нужном для ESP формате о наличии файла обновления, это было с расчётом на то, что устройство будет использовано на даче, а я мог обновлять его из города. Плюсы - есть, минусы - тоже. Схема такая: я разрабатываю ПО, создаю бинарник и кладу его на сервер, устройство опрашивает сервер о наличии обновления, скачивает и устанавливает его, со своей стороны. Удобно? В какой-то степени да, не надо прописывать IP (у меня две подсети), если мне потребовалось что-то поменять, я залил обновление на сервер и оно обновится, когда ему будет удобно.

Но возникла проблема - у меня сервер в OVH, какой-то антибот заблокировал подсеть и устройство перестало получать обновления (а также отправлять информацию), плана Б на такой случай я не предусмотрел.

Вторая проблема - неконтролируемое обновление, например, ошибка, залил скетч не от того устройства, либо залил нерабочий скетч - получил кирпич, а если на устройство была надежда (например, это полив), то придётся доставать отвёртку. Для себя сделал вывод, что для обновления скетча нужна копия устройства, нужно проверить обновление на нём, а затем загружать на устройство и делать это в контролируемой среде, то есть непосредственно на месте.

Корпус

Герметичность - с наступлением холодов муравьи сползли в корпус, потому что он теплее, да и в целом, насекомые насекомые и окислители туда могут пролезть, так что провода лучше продевать через герметичные соединения, стенки делать потвёрже и между крышкой и основанием добавлять прокладку: спроектировать посадочную канавку и вырезать прокладку.

Низкий уровень требования к печати - это ошибка, корпус должен защищать устройство. Конечно, SLA-печать даёт в разы лучший результат, я попросил друга распечатать один раз, получилось огненно, но у меня только дешёвый FDM, спускать ему его обычные погрехи не стоит, нужно как следует просушить филамент и докрутить настройки до максимально качественной печати, долго, но необходимо.

Электроника

Основные ошибки сводятся к неправильному использованию портов - нужно учитывать, что у них есть ограничение по току и есть состояние при включении. Также нужно помнить про питание, у меня возникали проблемы даже на отладочной плате, контроллер зависал, хотя внешне это не всегда понятно, поэтому для отладки и тестирования esp я ставлю лабораторный блок питания.

GPIO0: Этот пин должен быть HIGH (3.3V) для нормальной загрузки. Если он в момент включения оказывается на LOW (GND), модуль перейдет в режим прошивки. У модуля есть внутренняя подтяжка, но для максимальной стабильности нужен внешний резистор между GPIO0 и питанием 3.3V.

GPIO2: Также должен быть HIGH при загрузке. Если этот пин при старте в состоянии LOW, это может привести к непредсказуемому поведению модуля. На этом пине происходит “дребезг” при включении. В модуле с реле этот пин используется для включения, поэтому при подаче напряжения реле делает несколько стуков, что бы от этого избавиться я припаиваю конденсатор между GPIO2 и землёй.

GPIO1 (TX) и GPIO3 (RX): Эти пины используются для UART, при загрузке на TX может что-то появиться, поэтому

У esp01 в лоб можно использовать только один пин - GPIO3 (RX)

Объяснить с  SourceCraft

► [Как выглядит жизнь, когда ты поставил себя выше этого правила](#)

Для ESP-12: GPIO4, GPIO5 — самые надежные пины, подходят для реле и I2C устройств. GPIO12, GPIO13, GPIO14 — стандартные пины SPI шины, идеальны для датчиков. GPIO16 — пин общего назначения, используется для выхода из спящего режима (Deep Sleep).

4 пина

Мне нужно починить вентилятор в санузле, а раз мне нужно в него залезть, я попутно добавляю ему рассудительности.

У меня есть 4 пина и идея: подключить датчики движения, влажности, открытия двери и реле для включения вентилятора. Можно посадить ещё и включение света (для этого нужно от модуля датчика движения сделать дополнительную разводку на отдельное реле), но я не включаю свет по ночам, а вот гости включают, поэтому автоматическое освещение я делать не буду.

Вот модули: датчик движения, датчик влажности, реле, датчик открытия двери, esp01 с кроваткой и питание. Можно ли собрать из этого рабочую схему? Со сложностями да, в лоб подключая модули - нет. У esp01 при старте пины GPIO0 и GPIO2 должны быть в HIGH состоянии, датчики при включении могут быть в любом, реле должно быть в LOW, датчик влажности - единственный, с кем можно было бы иметь дело, подтянув его на HIGH, но нет, esp не любит и такое тоже, буду использовать esp12.

Логика работы такая: обнаружили движение, считываем состояние датчика двери, если дверь закрылась, то человек внутри, исключаем, что зашёл, например, открыть окно. Человек внутри, дверь закрылась, спустя некоторое время (t_1) просто включим вентилятор на некоторое время (t_2), но не раньше (t_3), чем дверь откроется и не дольше некоторого безопасного времени t_4 . Также, включим вентилятор, когда влажность в помещении достигла некоторого порогового значения h_1 учтём гистерезис h_2 и максимальное время работы t_5 за некоторый период t_6 .

ПО

В репозитории создаю папку с названием проекта latrine, добавляю ссылки на библиотеки, создаю скетч, всё это небольшой командой, которая также создаёт директории для схем и 3д-моделей

```
#!/bin/sh
PROJECT_NAME="$1"
cd "${0%/*}"
mkdir "$PROJECT_NAME"
cd "$PROJECT_NAME"
ln -s ../../lib/Zevice.cpp "./Zevice.cpp"
ln -s ../../lib/Zevice.h "./Zevice.h"
echo "#include \"Zevice.h\"\n\nZevice device;\n\nvoid setup() {\n\tdevice = Zev"
mkdir "$PROJECT_NAME"_schematics
mkdir -p "build/esp8266.esp8266.generic"
mkdir "$PROJECT_NAME"_3d
```

Объяснить с 

Начинаю программировать, вначале нужно определить константы, в коде использовать только их, что бы не путаться и менять сразу в одном месте. Значения сейчас можно

записать любые, я сосредоточен на логике. Названия константам лучше всего давать максимально понятные:

```
#define DHTTYPE    DHT11

#define PIN_MOVEMENT_DETECTOR 12
#define PIN_DOOR_DETECTOR 13
#define PIN_DHT 14
#define PIN_RELAY 4

#define T1_MIN_DOOR_CLOSED_DELAY 500
#define T2_TURN_ON_MAX_DURATION 500
#define T3_TURN_ON_MIN_DELAY_ON_OPEN 500
#define T4_TURN_ON_SAFE_DURATION 500
#define H1_HUMIDITY_ON 60
#define H2_HUMIDITY_OFF 50
#define T5_MAX_TURN_ON_FOR_PERIOD 500
#define T6_TURN_ON_PERIOD 500
```

Объяснить с 

Инициация:

```
pinMode(PIN_MOVEMENT_DETECTOR, INPUT);
pinMode(PIN_DOOR_DETECTOR, INPUT);
pinMode(PIN_DHT, INPUT);
pinMode(PIN_RELAY, OUTPUT);
```

Объяснить с 

И вспомогательная функция, которую я использую для асинхронных операций:

```
void onInterval(unsigned long &lastTime, unsigned long interval, void (*)());
void onInterval(unsigned long &lastTime, unsigned long interval, void (*function)) {
    if (millis() - lastTime > interval) {
        lastTime = millis();
        (*function)();
    }
}
```

Объяснить с 

С ней код loop становится более читаемый:

```
void loop() {
    device.loop();
    onInterval(DHT_lastUpdateTime, DHT_Interval, updateDHT);
}
```

Объяснить с 

Инструментарий выносится в отдельные блоки и его удобно переиспользовать:

```
// Обработка DHT11
unsigned long DHT_Interval = 60000;
unsigned long DHT_lastUpdateTime = 0;
void updateDHT();
float DHT_t = 0.0;
float DHT_h = 0.0;
DHT dht(PIN_DHT, DHTTYPE);

// ...

void updateDHT() {
    float newT = dht.readTemperature();
    if (!isnan(newT)) {
        DHT_t = newT;
    }
    float newH = dht.readHumidity();
    if (!isnan(newH)) {
        DHT_h = newH;
    }
}
```

Объяснить с 

Собственно, можно проверить работу устройства, прежде, чем писать основную логику. Для этого я собираю все блоки воедино с помощью проводов. Поскольку всё достаточно хлипко, удобно загружать прошивку по воздуху, поэтому я загружаю скетч вместе с библиотекой загрузки обновления. Сейчас мне нужно проверить, что все пины отработывают правильно и устройство стабильно включается, этого будет достаточно, что бы переходить к корпусу.

Базовый сетап esp12: припаять резистор между Vcc и EN, перемычку IO15-GND и провода для прошивки - TX, RX, GND, VCC, IO0. Перед тем, как вставить программатор в порт - прижать провод IO0 к GND. При загрузке скетча смотрю Mac-адрес чипа, затем в настройках роутера задаю ему статический адрес. После прошивки, не отсоединяя от

программатора отключаю-включаю USB, если по адресу /zevice/upload открывается форма загрузки обновления - можно отсоединять программатор.

```
#include "Zevice.h"
Zevice device;

void setup() {
    device = Zevice();
    device.autoConnect();
}

void loop() {
    device.loop();
}
```

Объяснить с 

Базовая библиотека

▸ [Zevice.h](#)

▸ [Zevice.cpp](#)

Хм, только сейчас заметил, что я планировал функцию переполнения millis, но так и не написал её.

Макетные платы и коннекторы — это отдельный ад, особенно в такой электронике. Помехи создают ложное понимание о работе устройства, поэтому чем туже соединяются коннекторы — тем лучше, а иногда лучше пропаять, чем вставлять в макетку, а макетку использовать для создания отдельного модуля (строго вход, выход и питание).

Необходимо проверить состояние датчика движения, он работает так: произошло движение - он активировался и активен какое-то время, поэтому я могу проверять его состояние, например, раз в секунду, нет необходимости проверять его каждый тик, тогда я добавлю ещё одну константу периода опроса датчика (можно было бы делать каждый тик, но у нас ещё работает веб-сервер, зачем добавлять нагрузку? С другой стороны, если человек торопится, то он успеет закрыть дверь за ту скорость, что быстрее скорости

света, но нам какая разница, мы же видим, что дверь закрыта, а значит решение однозначно).

Но дверь может и не закрываться, например, когда человек в доме один, то есть не следует полагаться исключительно на дверь, давайте так: если есть движение в течение какого-то периода T8, то мы подразумеваем, что кто-то внутри, но это T8 должно быть меньше времени активного состояния датчика (а этот параметр регулируется подстроечным резистором). Заведу константу, которая будет хранить время, которое будет считаться идентичным закрытию двери.

```
#define T7_MOTION_SENSOR_PERIOD 500
// ensure this value is shorter than movement sensor
#define T8_CONSEQUENT_MOVEMENTS_AS_CLOSED 500
```

Объяснить с 

Теперь код опроса:

```
// движения
unsigned long MS_Interval = 60000;
unsigned long MS_lastUpdateTime = 0;
void updateMS();
bool MS_state = false;
unsigned long MS_stateLastMillis = 0;

// loop
onInterval(MS_lastUpdateTime, MS_Interval, updateMS);

// updateMS
void updateMS() {
    MS_state = digitalRead(PIN_MOTION_SENSOR) == HIGH;
    MS_stateLastMillis = millis();
}
```

Объяснить с 

И код обработки смены состояния: когда сменилось состояние датчика движения на true, consequent movement будет означать, что на протяжении времени T8 мы увидим ещё движение, поэтому добавим переменные и одну константу для окна:

```
#define T9_CONSEQUENT_MOVEMENTS_LAG 5000
```

```

unsigned long MS_stateStart = 0;
unsigned long MS_stateEnd = 0;

MS_state = digitalRead(PIN_MOTION_SENSOR) == HIGH;
if (MS_state) {
    if ((MS_stateStart + T8_CONSEQUENT_MOVEMENTS_AS_CLOSED + T9_CONSEQUENT_MOVE
        MS_stateEnd = millis();
    } else {
        MS_stateStart = millis();
    }
}

if ((MS_stateEnd > MS_stateStart) && (MS_stateEnd - MS_stateStart) > T8_CONSEQU
    // Turn on the ventillation
}

```

Объяснить с 

И проверить состояние двери спустя это время

```

void updateDS() {
    bool doorIsClosed = digitalRead(PIN_DOOR_SENSOR) == LOW;

    // The door was opened
    if (DS_state && !doorIsClosed) {
        MS_stateStart = 0;
        MS_stateEnd = 0;
    }

    // The door was closed
    if (!DS_state && doorIsClosed) {
        if (MS_state) {
            // Turn on
        }
    }

    DS_state = doorIsClosed;
    DS_stateLastMillis = millis();
}

```

Объяснить с 

Включаем вентиляцию и определим, когда её отключить: не ранее, чем дверь откроется + некоторое время T10 или безопасное время работы, установим безопасное время, а в

другом потоке, при открытии двери, определим время отключения как минимальное между временем после открытия двери и безопасным временем работы

```
#define T10_TURN_OFF_AFTER_LEAVE 500
// ...
unsigned long turnOffVentilationAfter = 0;
void ventilationTurnOn() {
    if (ventilationIsTurnedOn) {
        return;
    }

    ventilationIsTurnedOn = true;
    turnOffVentilationAfter = millis() + T4_TURN_ON_SAFE_DURATION;
}

// Конструкция периодической проверки отключения вентилятора
unsigned long VS_Interval = 60000;
unsigned long VS_lastUpdateTime = 0;
void updateVS();
void updateVS() {
    if (ventilationIsTurnedOn && turnOffVentilationAfter < millis()) {
        ventilationIsTurnedOn = false;
    }
}

// ...
void updateDS() {
    // The door was opened
    if (DS_state && digitalRead(PIN_DOOR_DETECTOR) == LOW) {
        if (turnOffVentilationAfter > millis()) {
            turnOffVentilationAfter = std::min(millis() + T10_TURN_OFF_AFTER_LEAVE,
            );
        }
    }
    // ...
}
```

Объяснить с 

Также, включим вентилятор, когда влажность в помещении достигла некоторого порогового значения h_1 учтём гистерезис h_2 и максимальное время работы t_5 за некоторый период t_6 . Что бы не мешать логике срабатывания вентилятора с логикой увлажнителя, я сделаю отдельную функцию для включения вентилятора при избыточной влаге.

```

bool dehumidifierTurnedOn = false;
void updateDHT() {
    // get values
    if (DHT_h > H1_HUMIDITY_ON) {
        dehumidifierTurnedOn = true;
    }

    if (DHT_h < H2_HUMIDITY_OFF) {
        dehumidifierTurnedOn = false;
    }
}

```

Объяснить с 

Начало и конец ночи - в секундах от нулей, при этом ночь может быть как до нулей, так и после, поэтому учту и это тоже: сейчас значения в константах, но я хотел бы вынести это в конфигурацию.

```

#define T11_NIGHT_TIME_START 82800 // 23:00
#define T12_NIGHT_TIME_END 25200 // 07:00

const char* TZ_INFO = "MSK-3";

unsigned long NS_Interval = 60000;
unsigned long NS_lastUpdateTime = 0;
void updateNS();

// init
void setup() {
    configTime(TZ_INFO, "192.168.1.1", "pool.ntp.org");
    unsigned long check_until = millis() + 5000;
    while (time(nullptr) < 86400 && millis() < check_until) { // Check if valid
        delay(500);
    }
}

// loop
onInterval(NS_lastUpdateTime, NS_Interval, updateNS);

void updateNS() {
    time_t now = time(nullptr);
    struct tm* timeinfo = localtime(&now);
    int current_sec = timeinfo->tm_hour * 3600 + timeinfo->tm_min * 60 + timeinfo->tm_sec;
    if (T11_NIGHT_TIME_START < T12_NIGHT_TIME_END) {

```

```

        NS_state = current_sec > T11_NIGHT_TIME_START && current_sec < T12_NIGHT_TIME_START;
    } else {
        NS_state = current_sec > T11_NIGHT_TIME_START || current_sec < T12_NIGHT_TIME_START;
    }

    // Здесь можно сдвинуть счётчик следующей проверки
}

```

Объяснить с 

► Со сдвигом счётчика

Соответственно, команду включения выношу в отдельную функцию, которая будет принимать решение

```

unsigned long RS_Interval = 5000;
unsigned long RS_lastUpdateTime = 0;
void updateRS();

int relay_state = LOW;
void updateRS() {
    relay_state = NS_state ? LOW : (dehumidifierTurnedOn || ventilationIsTurnedOn);
    digitalWrite(PIN_RELAY, relay_state);
}

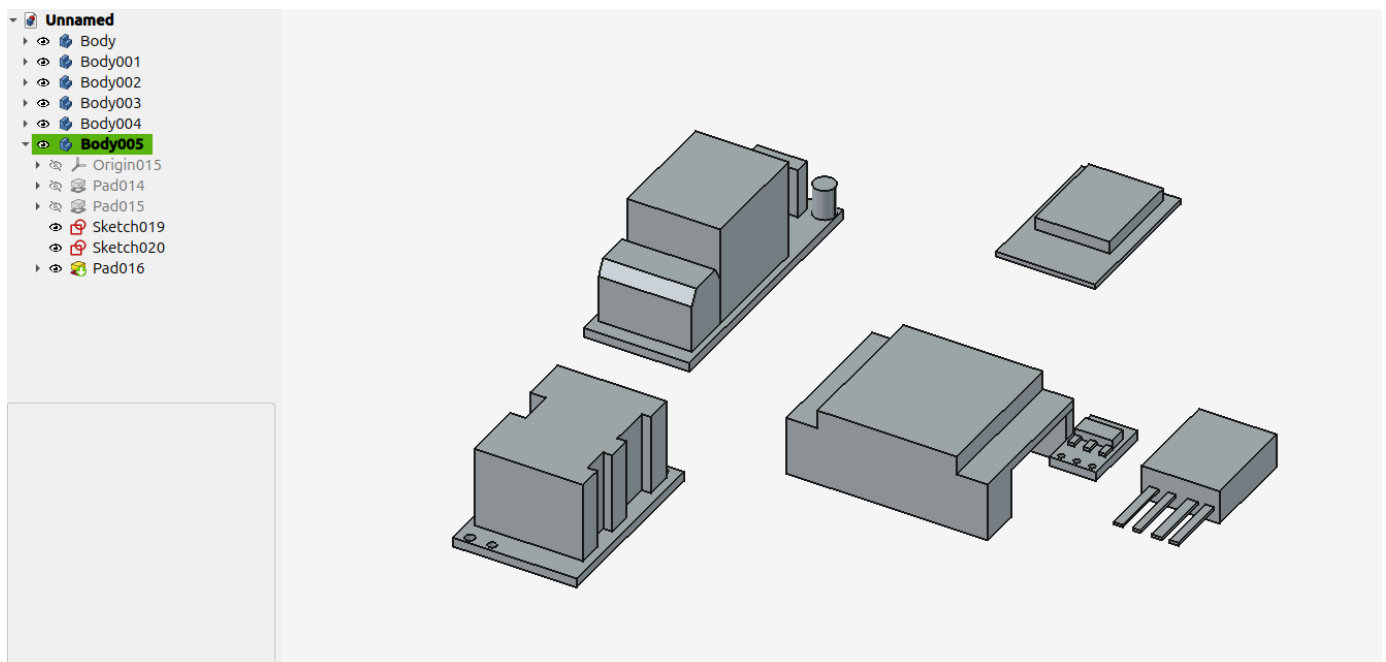
```

Объяснить с 

Пора собирать устройство, в целом тут нет необходимости в дополнительной плате, всю электронику можно паять прямо к esp12, поэтому я бы пропустил сейчас этот шаг и перешёл к проектированию корпуса, все элементы, которые мне нужны имеют известные размеры.

Проектирую корпус

Формат 3mf весит меньше всего, поэтому его и буду использовать для репозитория, так как от объектов мне нужны только габариты. Пока переносил размеры в модели обнаружил на нижней поверхности модуля питания посадочное место для AMS1117, отлично, одним блоком меньше, один модуль будет выдавать и 5v и 3.3v.

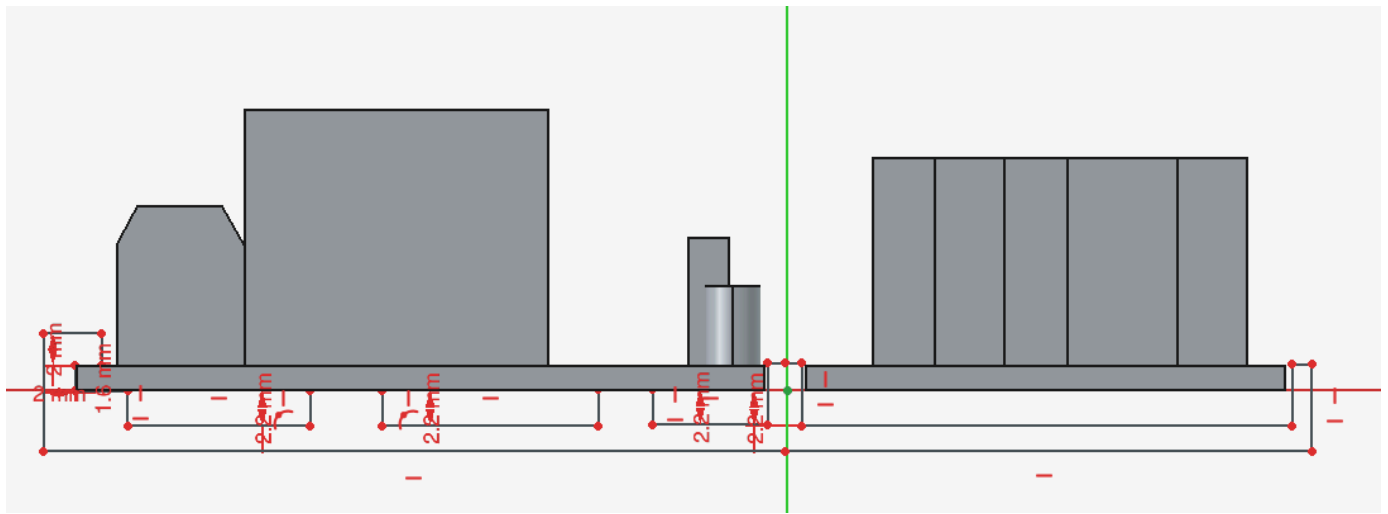


Габариты модулей

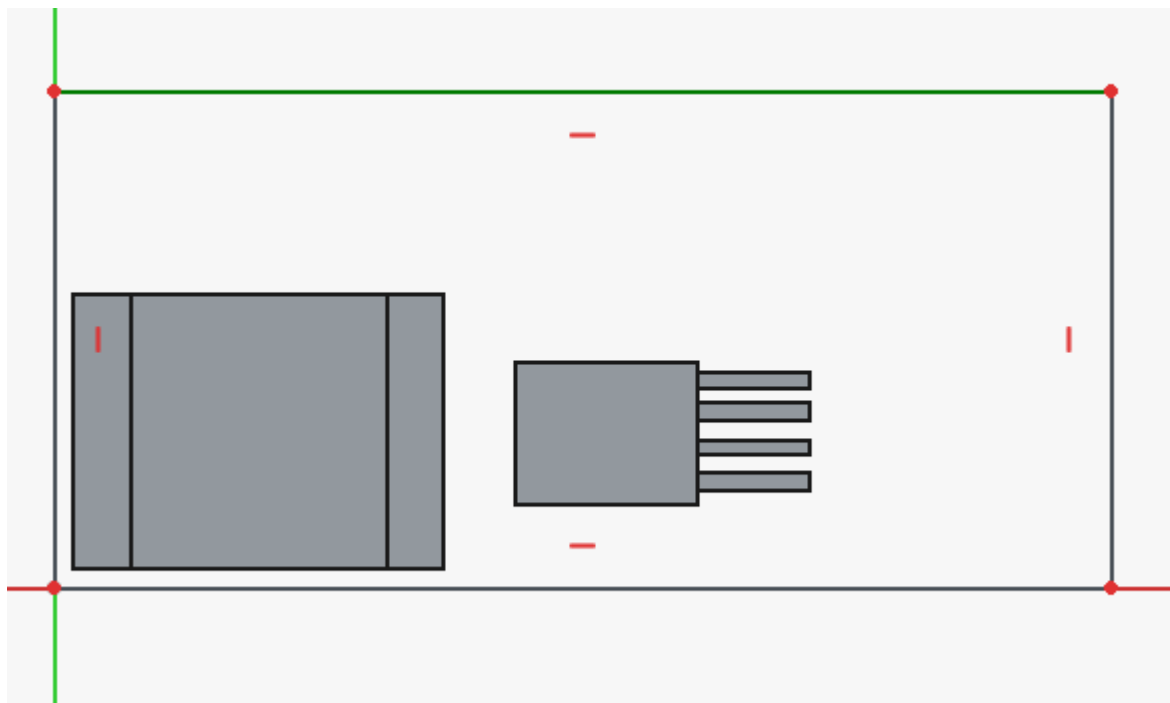
Нужно сделать так, чтобы все элементы были жёстко закреплены, изначально я делал так, что под каждый модуль рисовал рамку, а прижим осуществлял за счёт крышки. Здесь два модуля будут за пределами корпуса - это датчик влажности и датчик движения. У датчика движения есть отверстия диаметром 2мм, их можно использовать для винт-гайки, датчик влажности надо каким то образом прикрепить снаружи, наверное, это будут защёлки.

Внутри блоки могут идти друг за другом, а над ними будут располагаться датчики. Блоки питания и реле можно вставить по направляющим, модуль esp12... С одной стороны, провода не дадут ему прижаться к блоку питания, но скорее всего, неудобно будет заправлять, может быть поставить его вертикально? Тогда его можно вставить в крышку, но придётся провода делать длиннее, а это дополнительные перевязки внутри. Но датчики будут расположены на крышке, поэтому их можно провести там, тогда этой проблемы нет. Получается на основании два блока и три блока на крышке, переношу идею в 3D.

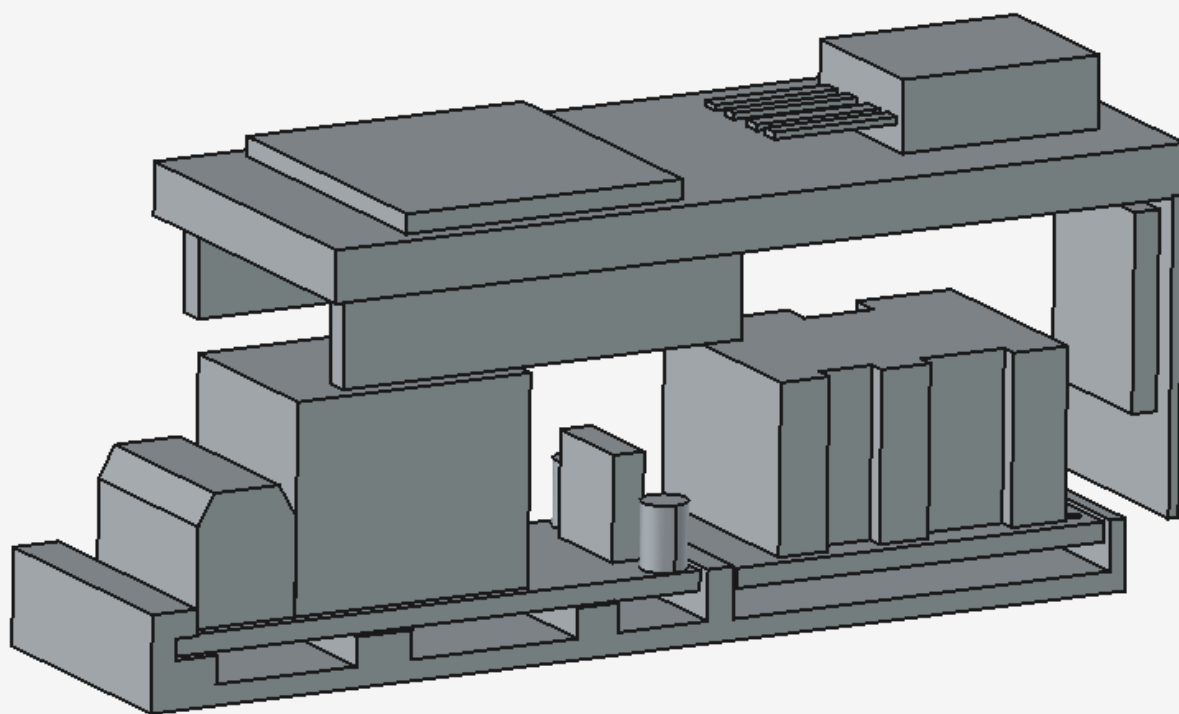
Для основания я продумываю профиль - создаю тело и рисую разрез, т.к. блок будет двигаться в этом направлении. Оставляю пазы для ножек. У блока питания всё настолько плотно, что прижаться там негде, поэтому я прижму его крышкой.



Для крышки - рисую поле с запасом, располагаю элементы, уменьшаю до нужного размера и оставляю небольшой запас справа для esp: ножки у DHT будут внутри, поэтому это место и есть запас. Вырезаю окно для датчика движения. Теперь можно DHT приблизить.

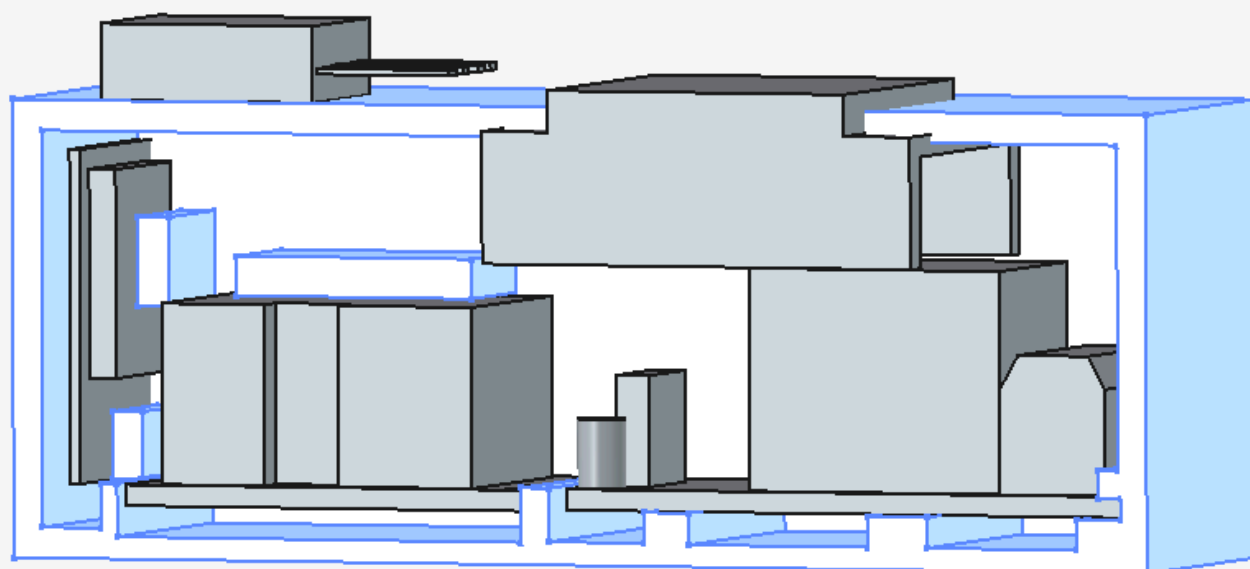


Теперь нужно сопоставить размер с основанием и понять как они будут соединяться. Левая стенка основания нам нужна для подключения проводов к реле, правую мы используем для esp, крышка должна прижимать трансформатор. С наскака придумать не получается, сопоставлю их в программе.



А вот так примерно это будет в сборе

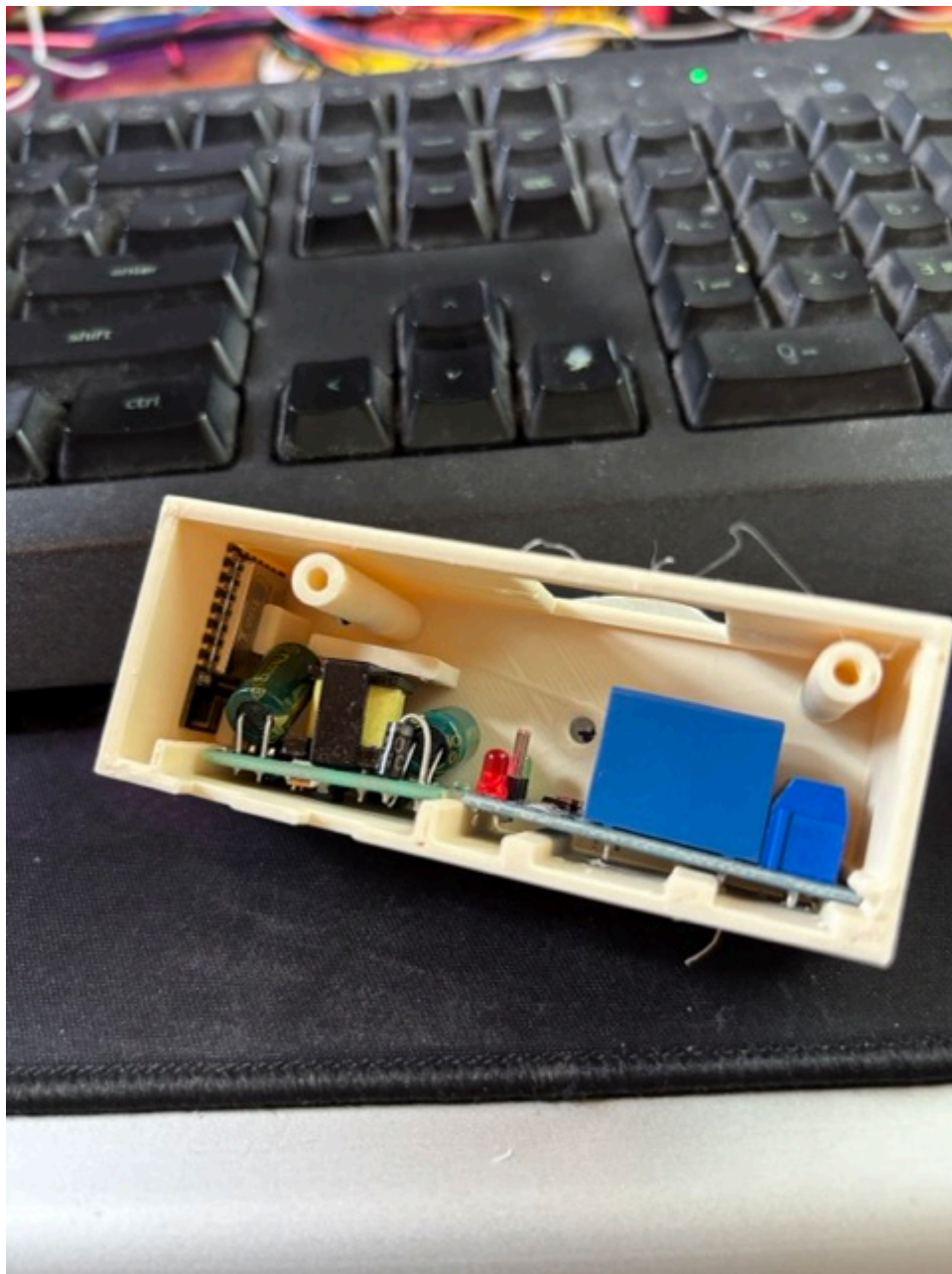
Пожалуй, я поменяю дно со стенкой, тогда модули удобнее вставлять - сначала вставляется датчик движения, прикручивается, затем по направляющим вставляются блоки esp, реле и питания, затем они прижимаются крышкой.



Обидно смотреть на такое количество пустого пространства, размер коробки получился ~30х30х90. Хотя может и не пустое, ещё ведь нужно место для присоединения проводов - контакт у реле мне не очень нравится в плане надёжности, поэтому я присоединю провода к нему и вынесу на колодку. Ещё не хватает крепления самой коробки к пространству, она хоть и не тяжёлая, но лучше бы её закрепить, смотрю на месте, как её расположу. Мне нравится вариант прикрутить её к стене через заднюю стенку, добавлю там отверстия.

Как заводить провода? Думаю вместе с крышкой через уплотнитель. Должны прийти два силовых и уйти два силовых, либо прийти два а уйти один, первый вариант мне нравится больше - меньше шансов на ошибку. Нагрузка будет небольшая, вентилятор это около 200 Вт (оказалось, 8 Вт), поэтому можно использовать самый тонкий провод, но они обычно 2 или 3 жилы, специальный я покупать не хочу, поэтому вариант с тремя проводами становится приоритетнее, посмотрю, есть ли у меня тонкий трёхжильный кабель. Нет, возвращаюсь к двум двухжильным. В наличии 4.5*6.5мм, он и будет прототипом.

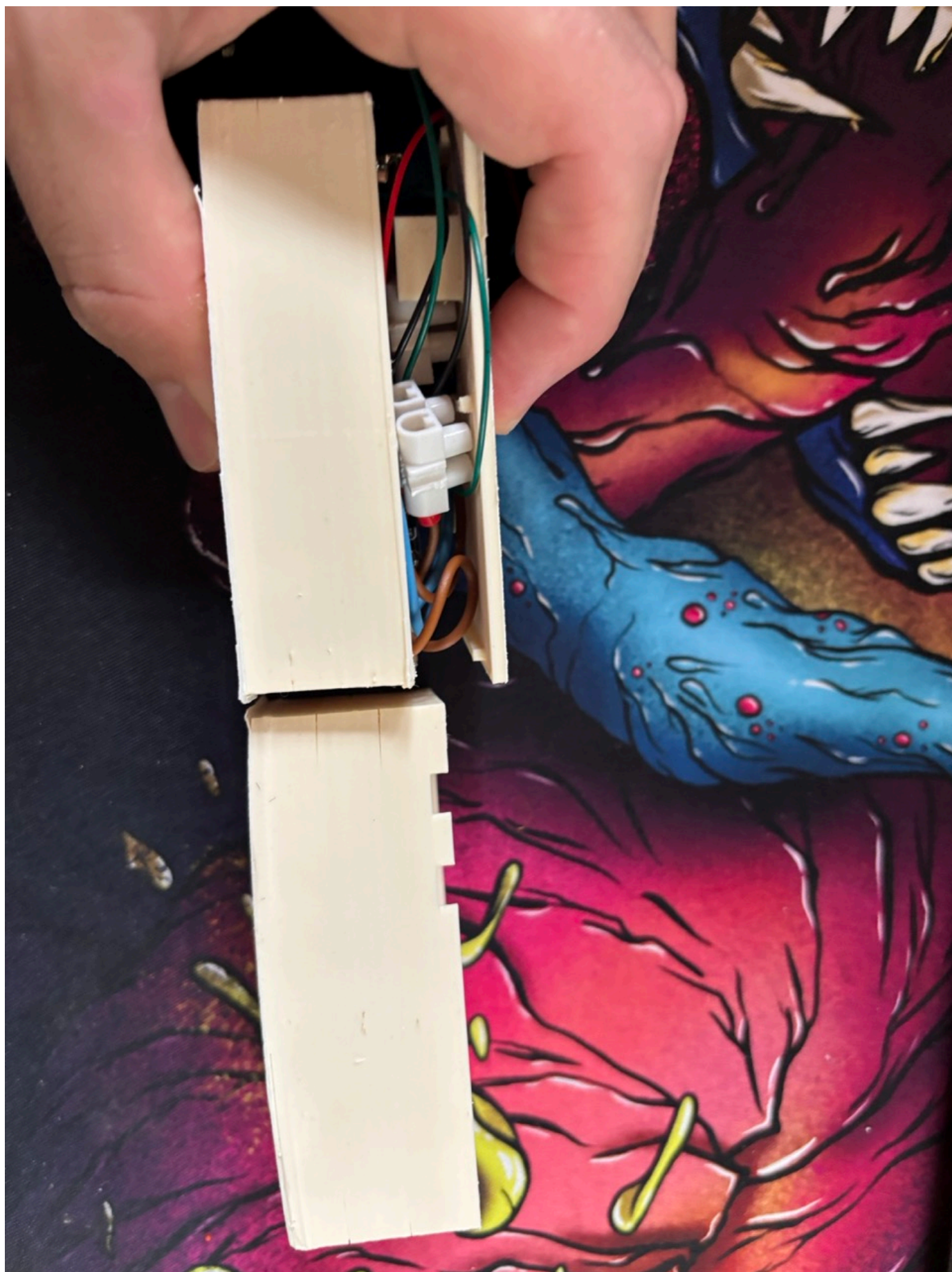
Пора печатать, буду использовать ABS-пластик, на стол намазываю раствор ABS в ацетоне, иначе деталь отклеивается спустя несколько слоёв. Печать корпуса и крышки - 3 часа, 30г филамента, достаточно экономно получилось.



Первая версия корпуса. Стойки от пола легко отламываются

Датчик движения не влезает, пришлось расширить для него границы.

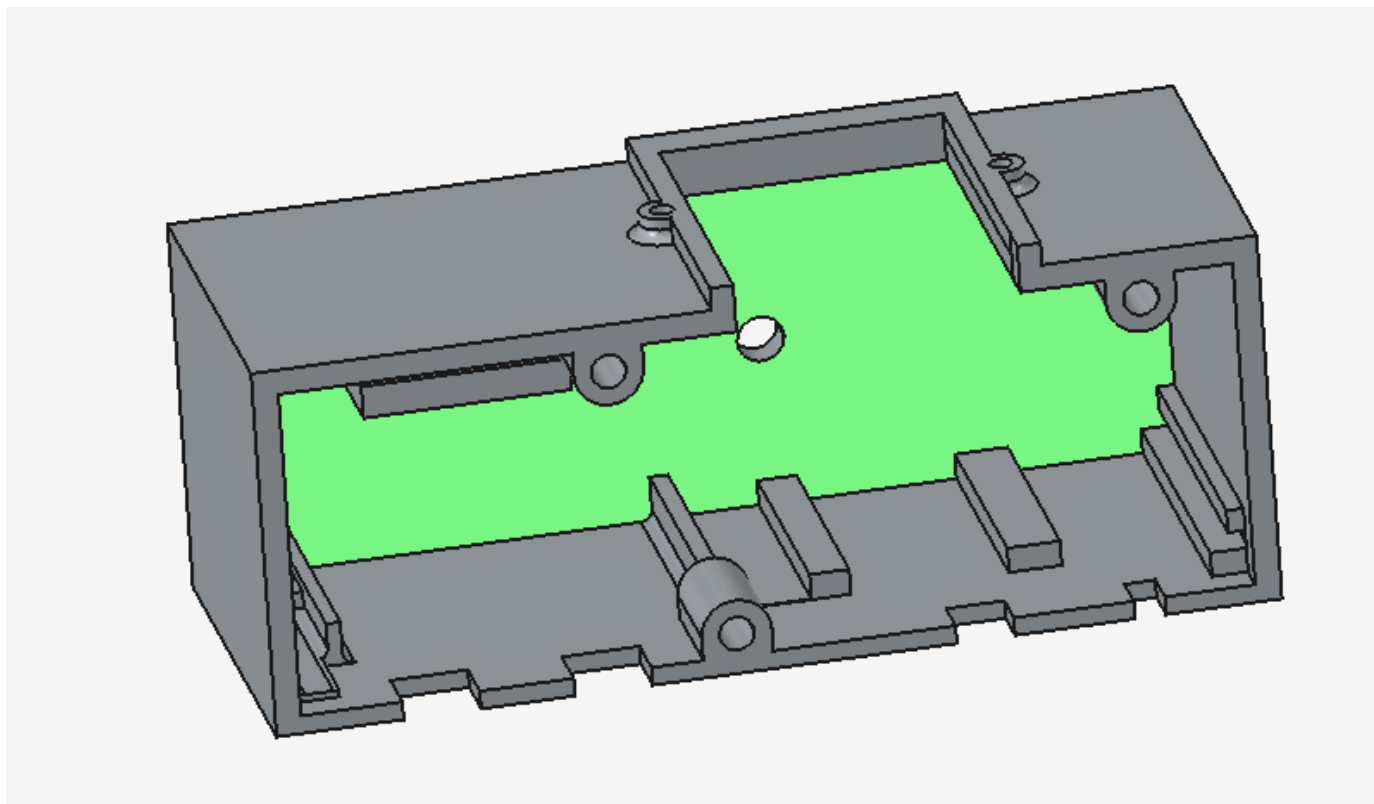
Волею погоды было отключено электричество на середине печати, повторный запуск провалился, так как я сдвинул ось Z, сложенная работа! Я перепроектировал корпус.



Что мы не учли так это то, что не учли

Теперь сюда добавились клеммы, им требуется много места, за это я их не люблю и поэтому всегда про них забываю, ну что же, пока печаталась вторая версия корпуса, свет отключился снова и у меня снова появился шанс перепроектировать с учётом всех

изменений - добавил объёма, третье крепление для крышки и отверстия для вывода датчиков.



Случайно подвернулась мысль закрепить на трубу, я взял крепление для трубы и оно отлично подошло, не знаю, как пойдёт, когда труба будет 60 градусов, но зимой всегда есть время на отладку!



В целом я доволен

Оно ведь не могло просто взять и заработать

Когда я всё установил - отказалось работать реле, не понимаю, почему оно работало на макетной плате, потому как ему требуется 5 вольт для включения, а у меня включался от 3.3, видимо от лабораторного блока напряжение было чуть выше, а сейчас в сети есть просадка и у меня не 220, в общем, сначала я грешил на пин (потому как реле щёлкало при старте), поэтому стал геминить, но гемини дал такой совет, что esp перестала подключаться к сети. В итоге пришлось снимать, разбирать и разбираться. Оказалось, что у модуля на входе PNP транзистор и он в принципе должен включаться при нуле и отключаться при 5v, но тестовый скетч щёлкания щелчка не дал, поэтому полез искать оптопару, нашёл 4n35, подключение простое и места в корпусе, слава погоде, достаточно.

Теперь всё запущено и работает, поскольку всё происходило около 23, то случайно проверил ночной переход - он работает. Вывел такую страничку для отладки:

WiFi.macAddress=40-91-51-46-2D-70

Temperature: 29.60

Humidity: 60.60

[Upload firmware](#)

The motion sensor state: Off

The door is open

The relay is turned off

Pins

PIN_MOTION_SENSOR PIN 13

PIN_DOOR_SENSOR PIN 14

PIN_DHT PIN 12

PIN_RELAY PIN 4

State

NS_state False

MS_state False

DS_state False

relay_state LOW

dehumidifierTurnedOn True

ventilationIsTurnedOn False

Отладочная информация модуля

Ну и на закуску

Оказалось, что вентилятор со встроенной логикой и он не включается, если влажность меньше 60%, 8 переключалок-настроек подали надежду, полез в мануал (ага, для вентилятора), можно прокинуть перемычку между L и Ls, тогда он будет включаться

каждый раз, когда ты его включаешь, он предусмотрен для автоматического включения вместе со светом, кабель нужно заводить третьим, но я не хочу лишний кабель, перемычка здесь сделает как раз то, что надо - когда будет подано напряжение - он включится, и здесь есть дополнительная защита, если реле, например, зависло, то он не будет работать дольше определённого времени.

Также, из мануала узнал про 50 секунд - время, которое не считается как необходимым для вентиляции.

Остались мелкие правки, но их можно сделать потом, сейчас всё работает и можно обновить по воздуху. Самое приятное - загружаешь прошивку, а оно не трещит и вентилятор не включается, потому что уже поздно, спать пора.

Теги: esp01, esp12

Хабы: Arduino, Электроника для начинающих, DIY или Сделай сам, Программирование микроконтроллеров, Умный дом

Редакторский дайджест

Присылаем лучшие статьи раз в месяц

Подписаться

Оставляя почту, я принимаю Политику конфиденциальности и даю согласие на получение рассылок



4K+

2

Охват за 30 дней

Карма

Zakhar Telyatnikov @za-ek2

Пользователь

Подписаться



Хабр Карьера

 Комментарии 5

Публикации

ЛУЧШИЕ ЗА СУТКИ ПОХОЖИЕ



monobogdan

22 часа назад

Полный разбор схемотехники кнопочного телефона. Samsung C100 — маленькое чудо корейской инженерной мысли



Простой



11 мин



15K

Ретроспектива



+46



26



43



runaway_llm

16 часов назад

Слишком просто, чтобы быть правдой? GPT-5.6 выдал доказательство 50-летней математической гипотезы

 Простой  4 мин  13K

Обзор

 +37

 12

 7



YuriPanchul
20 часов назад

Из обучения школьников цифре на макетке нужно выкинуть все резисторы и вставить микросхемы с FIFO

 Простой  3 мин  16K

 +31

 41

 138



oggr
6 часов назад

Кризис найма (как в IT) журналисты прошли ещё в 2013. Как у них получилось выйти?

 9 мин  7.6K

Мнение

 +29

 13

 27



Bright_Translate
3 часа назад

Басня о полусыром продукте

 Простой  11 мин  4.9K

Кейс

Перевод

 +24

 4

 1



Lunathecatt
23 часа назад

Лёгкий кастомный стратокастер из дешёвых комплектующих

 Простой  8 мин  11K

Тutorial

 +24

 4

 4



vital_pavlenko

1 час назад

Как я строил свой продукт, не увольняясь из найма. И что из этого вышло



6 мин



3.4K

Ретроспектива

 +16

 2

 7



Velibekov

16 часов назад

Как Яндекс.Навигатор может вести водителей в никуда или в туда, но не туда



Простой



6 мин



15K

Кейс

 +15

 10

 37



Geronom

19 часов назад

Бенчмаркая foreach и эnumераторы: аллокации, которые прятались 10 лет



Простой



17 мин



11K

 +15

 10

 3



Kova13v

23 часа назад

Я запретил Claude говорить «всё работает» без пруфов. Это изменило всё



Средний



9 мин



11K

Кейс

Из песочницы

 +15

 32

 21

Программируем дома: пишем о DIY-проектах, где пригодился код, и получаем призы

Турбо

Показать еще

КУРСЫ

Инженер по автоматизации

В любое время

1С программист

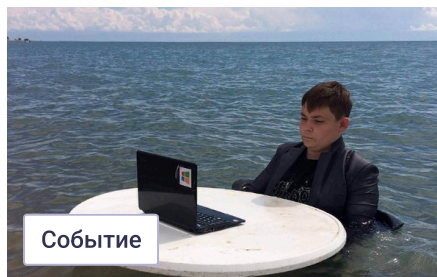
По мере набора группы

C++ для робототехников

По мере набора группы

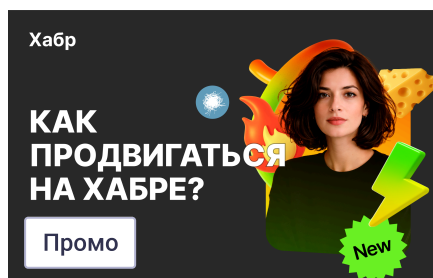
Больше курсов на Хабр Карьере

МИНУТОЧКУ ВНИМАНИЯ



Событие

Лето по расписанию: календарь событий Хабра



Хабр

КАК
ПРОДВИГАТЬСЯ
НА ХАБРЕ?

Промо

Экскурсия по Хабру для маркетологов



Data
Plane

Турбо

Пакеты
трафика

DPL в Overlay-слое: общение виртуальных машин

БЛИЖАЙШИЕ СОБЫТИЯ



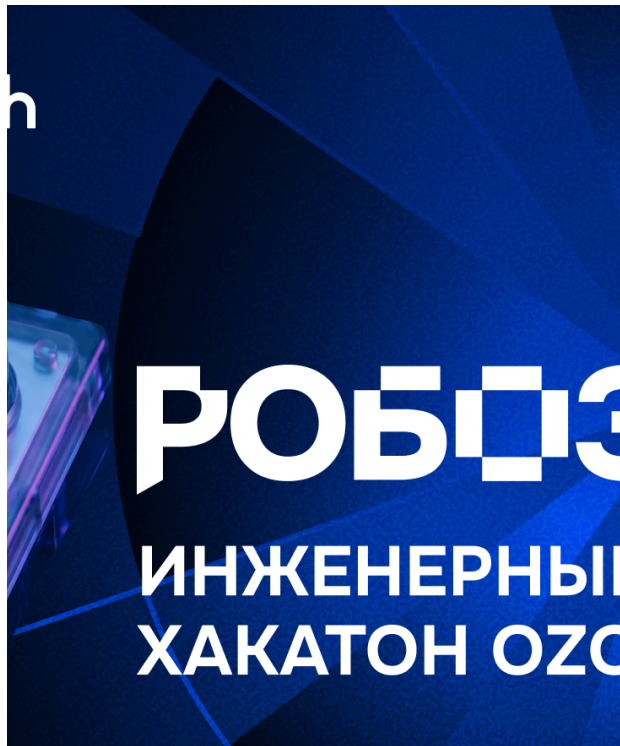
2 марта – 10 августа

**PROBA Awards —
международная
коммуникационная премия,
учрежденная в 2000 году**

Санкт-Петербург

[Маркетинг](#)

[Больше событий в календаре](#)



11 июня – 2 сентября

**Робозон: хакатон с призовым
фондом 15 млн руб.**

Москва • Онлайн

[Разработка](#)

[Больше событий в календаре](#)



15 июл

**Веби
эпох**

Онлайн

[Марк](#)

[Больше](#)



 [Настройка языка](#)

[Техническая поддержка](#)

